

ITC MPSI

TD exercice type concours

I) Autour des permutations (Épreuve mathématiques 2 de CCP 2024)

Pour $n \in \mathbb{N}^*$, on note S_n le groupe des permutations de l'ensemble $\llbracket 0, n-1 \rrbracket$. Une permutation de nS_n sera représentée en Python par une liste, dont l'élément d'indice i est l'image de i par cette permutation. Par exemple, la liste $[3, 1, 0, 2]$ représente la permutation $\sigma \in S_4$ définie par $\sigma(0) = 3, \sigma(1) = 1, \sigma(2) = 0, \sigma(3) = 2$.

Dans tout l'exercice, on pourra utiliser librement les tests Python du type `x in L` (respectivement `x not in L`) permettant de vérifier si x est présent dans la liste L (respectivement de vérifier si x n'est pas présent dans la liste L).

Q1) Si s est une liste Python représentant une permutation de S_4 , quelle instruction Python permet de trouver l'image de 1 par cette permutation ?

Quelle liste Python représente la transposition $(2\ 3) \in S_4$?

- `s[1]`
- `[0, 1, 3, 2]`

Q2) Écrire une fonction Python `comp(s1, s2)` prenant en entrée deux listes représentant des permutations σ_1 et σ_2 du même groupe de permutation et renvoyant la liste représentant la permutation $\sigma_1 \circ \sigma_2$.

```
def comp(s1, s2):  
    return [s1[x] for x in s2]
```

Q3) Écrire une fonction Python `inv(s)` prenant en entrée une liste représentant une permutation σ et renvoyant la liste représentant σ^{-1}

```
def inv(s):  
    sinv = s[:]  
    for i in range(s):  
        sinv[s[i]] = i  
    return sinv
```

II) Le problème de Freudenthal (partie ITC de l'épreuve d'option info de CCP 2020)

L'objectif de cette partie est de proposer une implémentation en langage Python d'une solution au problème de Freudenthal.

Hans Freudenthal (1905-1990), mathématicien allemand naturalisé néerlandais, spécialiste de topologie algébrique, est connu pour ses contributions à l'enseignement des mathématiques. En 1969, il soumet à une revue mathématique le problème suivant :

Énoncé 1

Un professeur dit à ses deux étudiants Sophie et Pierre : “J’ai choisi deux entiers x et y , tels que $1 < x < y$ et $x + y \leq n$. J’ai confié à Pierre la valeur Π du produit de x et y . J’ai confié à Sophie la valeur Σ de la somme de x et y . Pierre, Sophie, je vous demande de trouver x et y .”

Pierre et Sophie engagent alors le dialogue suivant :

- Pierre : “Je ne connais pas les nombres x et y .”
- Sophie : “Avant même que tu me le dises, je savais déjà que tu ne connaissais pas x et y .”
- Pierre : “Ah ! eh bien maintenant je connais x et y .”
- Sophie : “Très bien, mais moi aussi alors maintenant je connais x et y .”

Dans la suite, on note $N_n = \{(x, y) \in \mathbb{N}^2, 1 < x < y \text{ et } x + y \leq n\}$.

Si la discussion entre Sophie et Pierre semble stérile, une quantité importante d’informations est cependant échangée qui amène au bout du dialogue à la solution.

Q1) À quelle condition sur x et y Pierre aurait-il pu dire dès le début : “Je connais x et y ” ?

Si il n’existe qu’un seul couple d’entiers $(x, y) \in N_n$ tels que $xy = \Pi$, par exemple $\Pi = 10$.

[lien vers un notebook contenant le code corrigé](#)

Puisque Pierre ne peut répondre tout de suite, cela signifie que le produit Π peut s’écrire pour plusieurs couples d’entiers $(x, y) \in N_n$.

Q2) Écrire une fonction `CoupleProd(n)` qui renvoie la liste des entiers P pour lesquels il existe au moins deux couples $(x, y) \in N_n$ tels que $xy = P$. Par exemple, `CoupleProd(9)`=[12] puisque $12 = 3 \times 4 = 2 \times 6$ et qu’aucune autre valeur ne satisfait la propriété.

Sophie savait déjà que Pierre ne connaissait pas la réponse. C’est donc que, pour tout $(x, y) \in N_n$ qui satisfait $x + y = \Sigma$, le produit xy est dans la liste précédente.

Q3) Soit un entier $S \leq n$. Écrire une fonction `Prod(S, n)` qui renvoie, pour l’ensemble des $(x, y) \in N_n$ tels que $x + y = S$, la liste des entiers $P = xy$. Par exemple, `Prod(8, 9)` retourne [12, 15] puisque $8 = 6 + 2 = 3 + 5$.

Q4) Pour $S \leq n$, en déduire une fonction `Candidat_S(n)` qui renvoie la liste des entiers S tels que la liste `Prod(S, n)` est incluse dans la liste `CoupleProd(n)`

Pierre peut maintenant déduire la valeur de Σ du fait qu’elle appartient à la liste retournée par la fonction `Candidat_S(n)`. Plus précisément, le produit Π n’apparaît dans la liste `Prod(S, n)` que pour une seule valeur de S de la liste `Candidat_S(n)`. Pour déterminer cet unique S , on recherche tout d’abord les produits P pour lesquels :

- il existe deux sommes S_1 et S_2 dans la liste `Candidat_S(n)` telles que $S_1 < S_2$;
- P apparaît dans les listes `Prod(S_1, n)` et `Prod(S_2, n)`.

Q5) Écrire une fonction `Double_P(n)` qui renvoie la liste des produits P satisfaisant ces deux conditions.

Il reste à construire une fonction `Reste_S(n)` permettant de ne retenir que les sommes S de la liste `Candidat_S(n)` pour lesquelles il existe un unique élément de `Prod(S, n)` qui ne soit pas dans `Double_P(n)`.

Q6) Écrire une fonction `Reste_S(n)` qui renvoie la liste de ces sommes.

Pour que Pierre conclue, il faut que la liste $\text{Reste_S}(n)$ soit réduite à un singleton. Pour que Sophie conclue également, il lui suffit de rechercher les éléments de la liste $\text{Prod}(S, n)$ qui ne sont pas dans $\text{Double_P}(n)$.

Q7) Pour $S \leq n$, écrire une fonction $\text{Reste_P}(S, n)$ qui renvoie la liste de ces produits.

Les deux étudiants connaissent maintenant Σ et Π .

Q8) Écrire une fonction $\text{Solution}(n)$ qui retourne le couple (x, y) recherché.